EEST N°1 De Vicente López

Laboratorio de Programación (LPR) — 5° 3°A-B

Informe de Proyecto

Modelado de Datos con Estructuras (struct) y Punteros — Actividad 5

Alumno: Lucas Del Pino

Curso: 5° 3° A-B

Docente: Prof. York

Fecha de entrega: 25 de septiembre de 2026

Repositorio GitHub: https://github.com/Pin0l33/sistemaproyecto.git

1. Introducción

El presente informe documenta el desarrollo de la Actividad 5 de la materia Laboratorio de Programación (LPR), correspondiente al 1° cuatrimestre de 2026. El objetivo consistió en modelar una entidad del mundo real (sensor, módulo de hardware, usuario) mediante una estructura de datos (struct) en C++, controlando el ingreso y la modificación de sus miembros por referencia, mediante punteros y el operador flecha (->). El modelo sienta bases para las variables principales del Proyecto Integrador Anual.

2. Estructura del repositorio

El proyecto fue organizado en GitHub siguiendo la estructura solicitada por la cátedra:

```
/sistemaproyecto/ - .gitignore, LICENSE, README.md, docs/ (informe y
manuales/), src/, capturas/
```

La carpeta `docs/manuales/` contiene el manual del programador (versiones PDF y MD). El desarrollo se respaldó con commits descriptivos en las ramas `main` y `development`.

3. Captura del código fuente (Visual Studio Code)

La Figura 1 muestra el archivo `main.cpp` con el nombre real del alumno en el encabezado de la salida y en la inicialización de la estructura. La estructura `EntidadProyecto` tiene tres miembros: `id` (int), `nombre` (char[50]) y `metrica` (float). La función `cargarDatos` recibe un puntero a la estructura y completa sus campos con el operador flecha.

Figura 1

Código fuente main.cpp (Actividad 5)

```
int main() {
    ================================================================
    Materia: Laboratorio de Programación (LPR) – 5°
    Archivo: src/main.cpp
    Actividad 5: Modelado de datos con Structs y Punteros
    Alumno: Lucas Del Pino
    ================================================================
*/

#include <iostream>
#include <cstring>

using namespace std;

// 1. DEFINICIÓN DE LA ESTRUCTURA (Modelo sugerido para el Proyecto Integrador Anual)
struct EntidadProyecto {
    int id;
    char nombre[50];
    float metrica; // Representa lecturas de sensores, consumo, avance, etc.
};

// Prototipo de función con pasaje por dirección mediante puntero
void cargarDatos(EntidadProyecto* ptr);

int main() {
    // Inicialización de seguridad en la memoria Stack
    EntidadProyecto miEntidad = {0, "Vacio - Lucas Del Pino", 0.0f};

    cout << "======================================================" << endl;
    cout << "  MODELADO STRUCT - ESTUDIANTE: Lucas Del Pino " << endl;
    cout << "======================================================" << endl;

    // Invocación enviando la dirección física de memoria con '&'
    cargarDatos(&miEntidad);

    cout << "\n=== DATOS VERIFICADOS EN LA MEMORIA RAM ===" << endl;
    cout << "ID Registrado:     " << miEntidad.id << endl;
    cout << "Nombre Registrado: " << miEntidad.nombre << endl;
    cout << "Metrica Guardada:  " << miEntidad.metrica << endl;
    cout << "Direccion RAM Hexadecimal: " << &miEntidad << endl;
    cout << "======================================================" << endl;

    return 0; // Código 0: Finalización exitosa
}

void cargarDatos(EntidadProyecto* ptr) {
    cout << "\n--- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA ---" << endl;
```

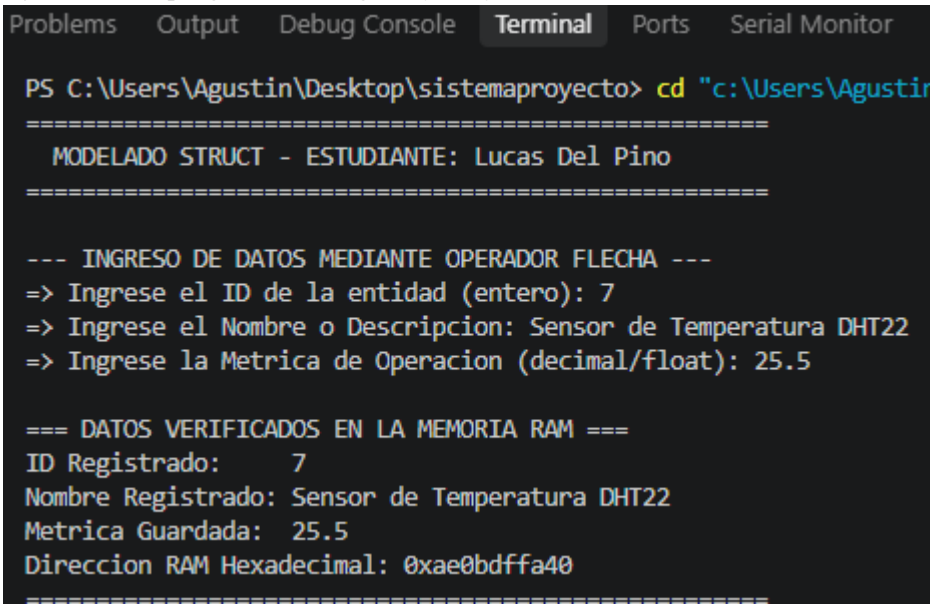
4. Captura de la ejecución (Terminal)

4.1 Compilación y ejecución del programa en C++

Se compiló con `g++ src/main.cpp -o src/sistemaproyecto.exe` (sin advertencias) y se ejecutó con `.\src\sistemaproyecto.exe`. Se ingresó ID 7, nombre "Sensor de Temperatura DHT22" y métrica 25.5; los datos se verificaron luego en pantalla (Figura 2).

Figura 2

Ejecución del programa con ingreso y verificación de datos

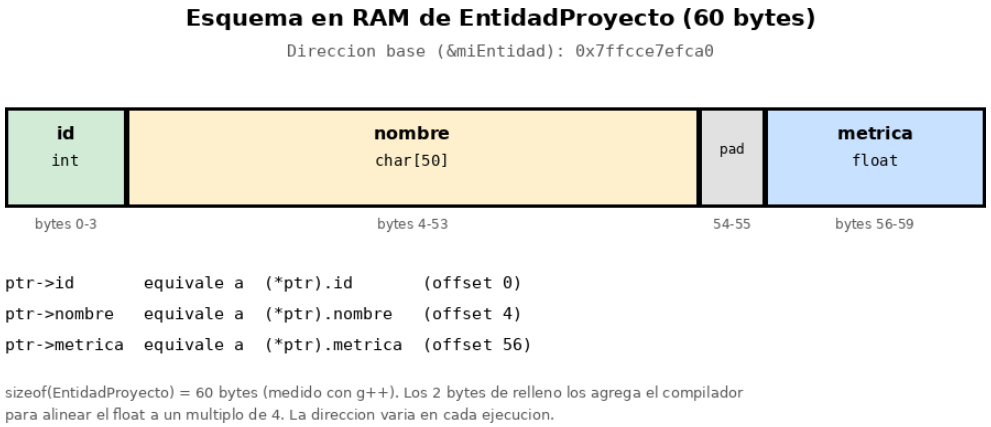


4.2 Esquema de la estructura en memoria

La Figura 3 muestra la disposición de los miembros en RAM. El tamaño medido con `sizeof` fue de 60 bytes: 4 del `int`, 50 del arreglo, 2 de relleno que agrega el compilador y 4 del `float`.

Figura 3

Disposición del struct EntidadProyecto en la memoria RAM



5. Cuestionario técnico de control

Pregunta 1: Heterogeneidad

¿Qué ventaja ofrece agrupar los datos de un sensor o dispositivo en un struct en lugar de declarar muchas variables sueltas?

El struct reúne bajo un solo nombre datos de tipos distintos (`int`, `char[]`, `float`) que pertenecen a la misma entidad. Esto ordena el código, evita mezclar datos de dispositivos diferentes, permite pasar toda la entidad a una función con un único parámetro y facilita ampliar el modelo agregando un miembro. Además, sus miembros ocupan bloques contiguos de memoria. Con variables sueltas habría

que manejar cada dato por separado y el código sería más propenso a errores.

Pregunta 2: Operador flecha

¿Por qué el compilador de C++ no permite usar la notación de punto (`ptr.id`) cuando la variable es de tipo puntero?

Porque el operador punto accede a los miembros de un objeto, y `ptr` no es la estructura sino una variable que guarda su dirección; un puntero no tiene miembros propios. Para llegar a la estructura hay que desreferenciar primero: `(*ptr).id`. El operador flecha `ptr->id` es la forma abreviada de esa operación, que desreferencia y accede al miembro en un solo paso.

Pregunta 3: Eficiencia en RAM

¿Por qué el pasaje por referencia mediante punteros optimiza la memoria procesada frente al pasaje por copia?

En el pasaje por copia se duplica toda la estructura en la pila (en este caso 60 bytes) cada vez que se llama a la función, y los cambios afectan solo a la copia. Con el pasaje por dirección se envía únicamente el puntero (por ejemplo 8 bytes en un sistema de 64 bits), sin duplicar datos, y la función modifica el original. La diferencia crece cuanto más grande sea la estructura.

Pregunta 4: Buffer de entrada

¿Qué comportamiento erróneo ocurre en la terminal si se omite la instrucción `cin.ignore()` antes de un getline?

Después de `cin >> ptr->id` queda en el buffer el Enter (`\n`) que el usuario presionó. Si no se lo descarta, `cin.getline` lo lee de inmediato como una línea vacía y termina sin esperar el texto: el programa se saltea el ingreso del nombre y pasa directamente a pedir la métrica, dejando el campo `nombre` vacío.

Pregunta 5: Aplicación al Proyecto Integrador

¿Qué entidad o componente hardware de su Proyecto Integrador Anual representaron con esta estructura y qué variables componen sus miembros?

No contamos con una entidad o componente hardware que represente esta estructura.

.

6. Sección personal: struct vs. variables sueltas

Trabajar con un struct me permitió ver una entidad como una unidad: en vez de tres variables independientes, hay un único objeto `miEntidad` cuya dirección se envía a una función. Entendí también que `ptr->campo` y `(*ptr).campo` son lo mismo, y que el orden de `cin`, `cin.ignore()` y `getline` importa por el buffer de entrada.

7. Conclusión

La actividad permitió modelar una entidad con una estructura de datos, modificarla por referencia mediante punteros y el operador flecha, y comprender el manejo del buffer de entrada. El programa funcionó correctamente en la prueba realizada y sienta una base reutilizable para el Proyecto Integrador Anual.